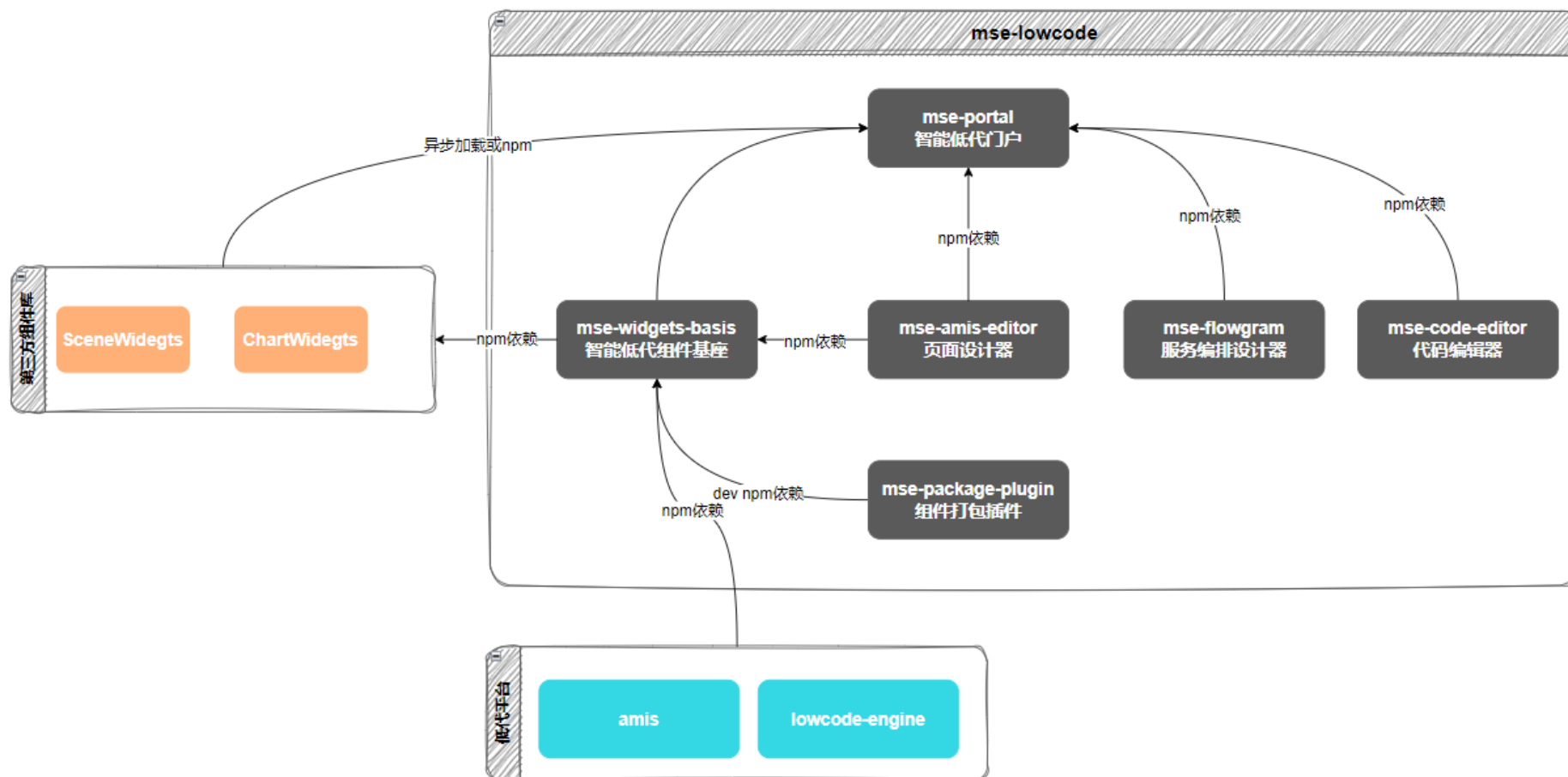


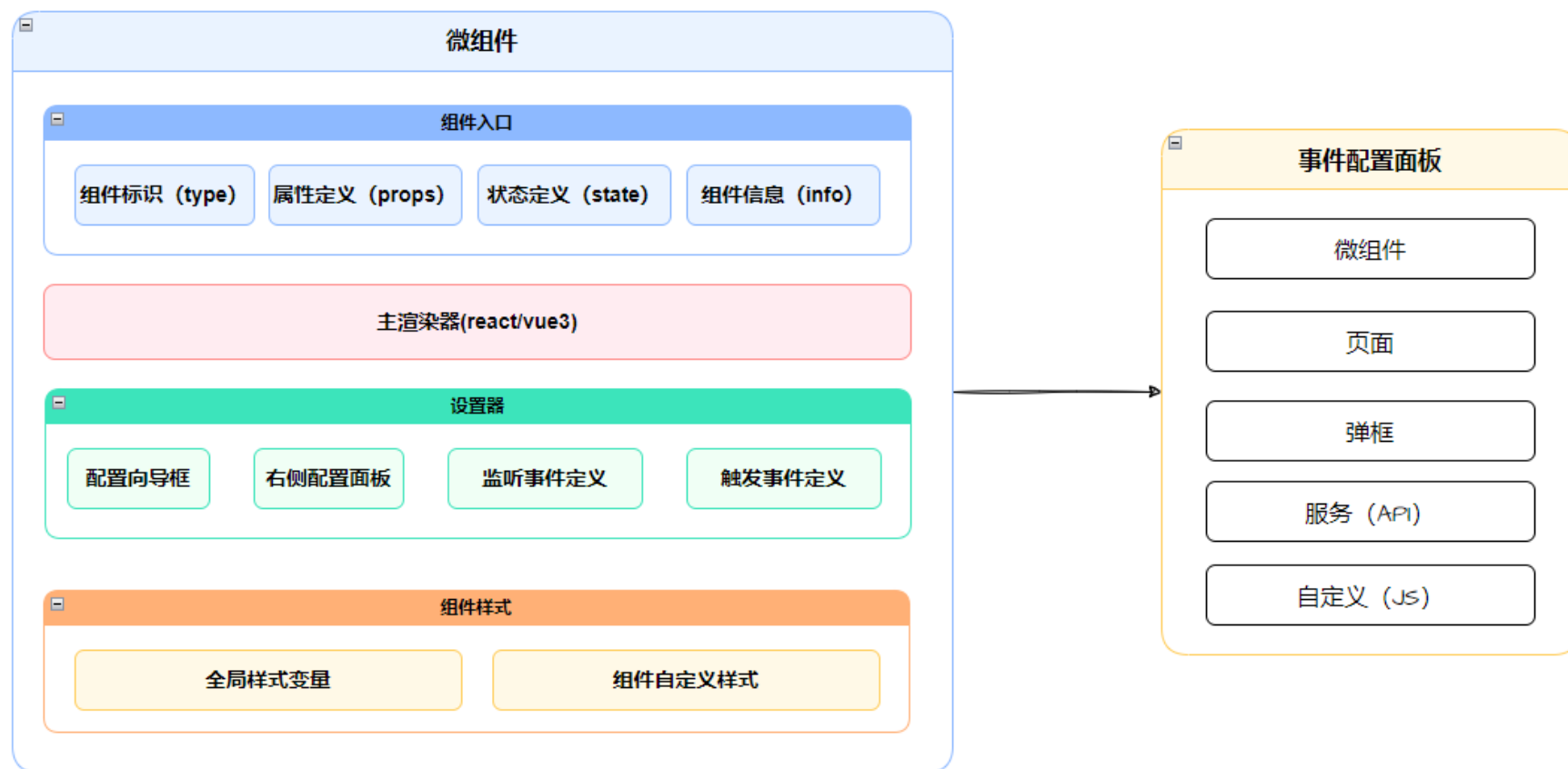
微前端组件（SuperMap-Micro-Widget）规范（初稿）

设计

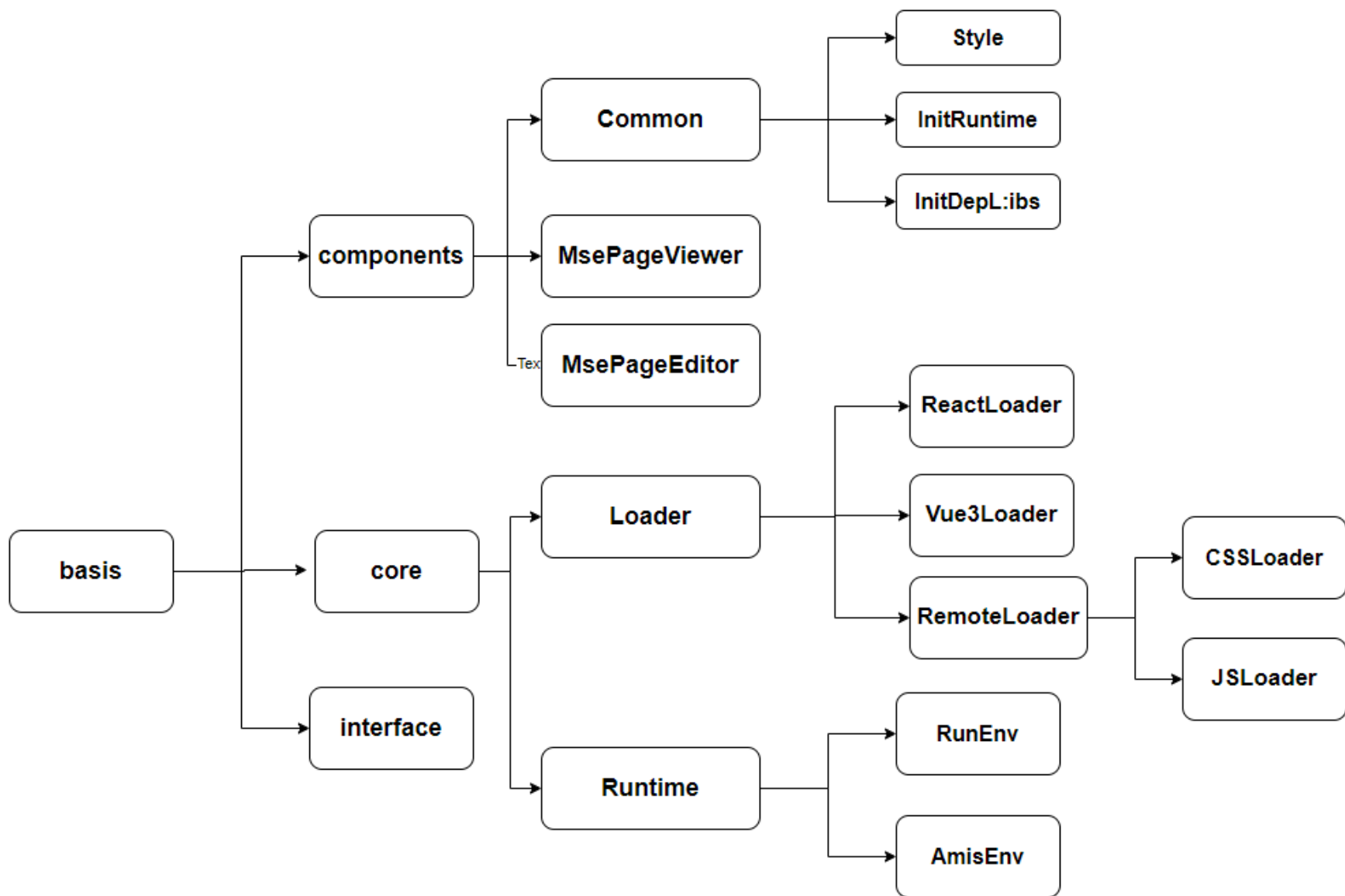
1、 整体设计



2、 微组件设计



3、 基座设计



一、 工程结构

1、 目录规范

规范化的目录结构是保证项目可维护性和工具链可交互性的基础。建议采用如下单仓库多组件的结构。

smw-widget-demo

| README.md(说明文档)

└─public

└─src/widgets

| └─assets (前端资源)

| └─common (组件库公共方法)

| └─component-a

| └─index.ts (组件入口)

| └─widget.tsx/.vue (组件主渲染器)

| └─config.tsx (组件配置面板和事件面板)

| └─style.scss (组件样式)

└─src (开发工程基础环境)

| └─App.tsx

| └─main.tsx

| └─index.scss

| └─vite-env.d.ts

└─vite.config.js

└─package.json

2、README.md

应该包含业务组件的源信息、使用说明以及 API，示例如下：

```
# 按钮 (组件中文标题)

按钮用于开始一个即时操作。 (组件功能描述)

## 安装方法 (此部分通常由构建工具自动生成)

npm install @smw-lowcode/custom-widgets -S

## API (属性配置说明，是低代码描述中 props 的具体化)

| 参数 | 说明 | 类型 | 可选值 | 默认值 |
| ----- | ----- | ----- | ----- | ----- |
| type | 按钮类型 | String | `primary`, `normal`, `danger` | `normal` |
| text | 按钮文本 | String | - | `按钮` |
| onClick | 点击事件回调 | Function | - | - |

更新说明
```


3、 **package.json**

package.json 中包含了一些依赖信息和配置信息，示例如下：

```
{
  "name": "@smw-lowcode/custom-widgets",
  "description": "smw 业务自定义低代码组件库",
  "version": "0.0.1",
  "main": "dist/index.js",
  "stylePath": "dist/style.css",
  "files": [
    "README.md",
    "dist/"
  ],
  "dependencies": {
    "antd": "5.x"
  },
  "devDependencies": {
    "react": "^18.0.0",
    "react-dom": "^18.0.0"
  },
  "peerDependencies": {
    "react": "^18.0.0",
    "react-dom": "^18.0.0"
  },
  "componentConfig": {
    "name": "smw-custom-widgets",
    "title": "smw 业务组件",
    "category": "business"
  }
}
```

版本说明

二、 组件库（资产包）

组件库（资产包）是将散落的组件、插件和资源组织成一个整体，供低代码引擎加载的集合。

1、 assets.json

这是组件库的核心清单，由打包工具中的插件自动生成，声明了组件库的元信息、包含的组件、配置器（setter）和插件等。

关键字段说明：

- name:组件库名称（统一使用中文）
- code:组件库唯一标识（统一使用英文）
- version： 组件库版本
- description： 组件库描述
- packages： 组件库依赖（npm 或其他第三方依赖）
- widgets： 组件列表

```
{
  "name": "示例组件库",
  "code": "smwWidgetDemo",
  "version": "1.3.1",
  "description": "smw 示例组件",
  "packages": [
    {
      "name": "超图 3D 资源",
      "code": "SuperMap3D",
      "version": "1.0.1",
      "type": "script",
      "path": [
        "/widgets/config.js",
        "/widgets/aa.css"
      ]
    }
  ],
  "widgets": [
    {
      "name": "React 示例组件",
      "code": "smw-react-demo",
      "description": "React 示例组件",
      "icon": "fa fa-user",
      "framework": "react"
    }
  ],
```

2、 入口文件 index.ts

该文件是组件库的 UMD 打包入口，负责将组件和其低代码描述一并导出。引擎通过该文件获取组件的运行时实体和配置描述。

```
import {Border1Widget} from './border_1'
import Package from '../package.json'
import type {LibraryInfo} from '@smw-lowcode/smw-widget-basis'

const libraryInfo: LibraryInfo = {
  name: '测试组件库',
  code: Package.name,
  description: Package.description,
  version: Package.version,
  widgets: [
    Border1Widget
  ]
}

export default libraryInfo
```

3、 第三方依赖资源

依赖资源元数据：

- name：资源名称

- version: 资源版本
- code: 资源唯一标识
- loadType: 加载方式 (npm 或 script)
- urls: 加载路径 (可以动态解析公共变量)

a、npm 依赖示例

对于外部依赖 (如 Ant Design、Lodash), 应在 assets.json 的 packages 项中声明, 并通过 urls 指定其 CDN 地址, 以避免重复打包和版本冲突, 低代码引擎会统一管理这些依赖的加载。

```
{
  "packages": [
    {
      "name": "antd", // 中文别名
      "version": "5.12.0",
      "library": "antd",
      "type": "npm",
      "urls": [
        "https://unpkg.com/antd@5.12.0/dist/antd.min.js",
        "https://unpkg.com/antd@5.12.0/dist/antd.min.css"
      ]
    }
  ]
}
```

b、<script>路径依赖

对于有些无法通过 npm 安装到工程里面，统一打包的依赖，可以通过动态<script>标签加载。

```
{
  "packages": [
    {
      "name": "SuperMap3D",
      "version": "5.12.0",
      "library": "SuperMap3D",
      "type": "script",
      "urls": [
        "https://unpkg.com/SuperMap3D@5.12.0/dist/SuperMap3D.min.js",
        "https://unpkg.com/SuperMap3D@5.12.0/dist/SuperMap3D.min.css"
      ]
    }
  ]
}
```

三、 组件（物料）

组件是资产包的基本单元，其开发需要遵循“配置驱动”和“高内聚”的原。

1、 组件入口(index.ts)

此文件是单个组件的入口，负责导出组件的 React 或 Vue3 实现 和其 低代码物料描述 。

定义并导出内容：

- IProps：组件配置属性
- IState：组件状态属性
- WidgetCode：组件唯一标识
- WidgetInfo：组件元数据信息


```
import type {WidgetInfo} from '@smw-lowcode/smw-widget-basis'
import widget from './widget'
import {Border1WidgetPlugin} from './config'
```

```
// 组件编码（组件库内保持唯一）
```

```
export const WidgetCode = 'border_1'
```

```
// 组件 props 定义
```

```
export interface IProps {
  w: number
  h: number
  backgroundColor: string
  dur: number
  colors: string[],
  body: any,
  render: any
}
```

```
// 组件 State 定义
```

```
export interface IState {
  loading:boolean
}
```

```
// 组件信息
```

```
export const Border1Widget: WidgetInfo = {
```

2、 渲染器

渲染器是组件的核心，负责根据配置属性（props）进行 UI 渲染。它必须是一个纯函数式的 React 组件，其行为完全由 props 驱动

- `Vue3(widget.vue)`

```
<script setup lang="ts">
import {ref, onMounted} from 'vue'
import {smwEvent} from '@smw-lowcode/smw-widget-basis'
import {IProps} from 'index.ts'

// 组件状态

const state = ref<Istate>({...})


// 组件属性

const props = defineProps<IProps>({...})


// 组件触发事件

const onClick = () => {
  console.log('btn click...')
  props.smwEvent.emit('click', {anb: 123})
}


// 组件监听事件

onMounted(() => {
  props.smwEvent.on('message', (args: any) => {
    console.log(args)
    alert('监听到事件触发')
  })
})
</script>
```

- `React(widget.tsx)`

```
import React from 'react'
import type {RendererProps} from 'smw-core'
import {ScopedContext} from 'smw-core'
import './style.css'

export default class LeafletWidget extends React.Component<IProps,IState> {
  static contextType = ScopedContext

  constructor(props: RendererProps) {
    super(props)
    this.props = props
    this.handleClick = this.handleClick.bind(this)
  }

  componentDidMount() {
    this.context?.registerComponent(this)
  }

  componentWillUnmount() {
    this.context?.unRegisterComponent(this)
  }

  doAction(action: any, args: any) {
    const actionType = action ? action.actionType : ''

    if (actionType === 'message') {
```

3、 配置面板（setter）

组件配置相关属性定义：

- 初始化钩子（组件预览、组件拖入编辑器初始化配置）
- 组件右侧配置面板结构定义（schema、schemaTpl）
- 监听事件定义
- 触发事件定义

完整示例如下：

```
export class ReactWidgetPlugin extends BasePlugin {

  // 用来生成预览图的

  previewSchema = {
    type: 'smw-react-demo',
    target: 'demo'
  };

  // 拖入组件里面的初始数据

  scaffold = {
    type: 'smw-react-demo',
    target: '233'
  };

  panelBodyCreator = (context: BaseEventContext) => {
    return (
      // 配置面板定义
    )
  };

  // 触发事件定义

  events: RendererPluginEvent[] = [{
    eventLabel: '按钮点击',
    eventName: 'click'
  }]

  // 监听事件定义

  actions: RendererPluginAction[] = [
    {
```

3.1、配置面板组件使用

1、 输入框

说明： 文本输入框，支持普通文本、密码、URL、邮箱等多种内容输入

示例

```
{  
  type: 'input-text',  
  label: '标题',  
  name: 'title'  
}
```

2、 数字输入框

说明： 限制输入为数字，可设置最大和最小值

示例


```
{
  type: "input-number",
  name: "w",
  label: "宽度",
  min: 0,
  max: 100
  value: 55,
}
```

3、选择框

说明：选择给定的列表

示例

```
{
  type: 'select',
  label: '位置',
  name: 'xAxisStyle.position',
  options: [
    {
      label: '底部',
      value: 'bottom'
    },
    {
      label: '顶部',
      value: 'top'
    }
  ]
}
```

4、文本域

说明：支持换行输入

示例

```
{
  "type": "textarea",
  "label": "多行文本",
  "name": "textarea",
  "minRows": 3,
  "maxRows": 20
}
```

5、 开关

说明：支持设定最大值和最小值，以及步长与精度

示例

```
{
  type: 'switch',
  label: '可见',
  name: 'enabled'
}
```

6、 颜色选择框

说明：支持 hex、hexa、hls、rgb、rgba 格式，默认为 hex 格式

示例

```
{  
  "type": "input-color",  
  "label": "颜色",  
  "name": "color",  
  "format": "hex"  
}
```

7、复选框

说明：通过 options 配置多个勾选框，也可以通过 source 拉取选项

示例

```
{
  "type": "checkboxes",
  "label": "复选框",
  "name": "checkboxes",
  "multiple": true,
  "options": [
    {
      "label": "选项 A",
      "value": "A"
    },
    {
      "label": "选项 B",
      "value": "B"
    }
  ],
  "checkAll": false,
  "joinValues": true,
  "optionType": "default"
}
```

8、 单选框

说明： 选择单个选项

示例

```
{
  "type": "radios",
  "label": "单选框",
  "name": "radios",
  "options": [
    {
      "label": "选项 A",
      "value": "A"
    },
    {
      "label": "选项 B",
      "value": "B"
    }
  ],
  "optionType": "default"
}
```

9、按钮

说明：用来展示一个按钮

示例

```
{  
  "type": "button",  
  "label": "按钮",  
}
```

10、选项卡

说明：选项卡，可以将内容分组用选项卡的形式展示

示例

```
getSchemaTpl('tabs', [  
  {  
    title: '属性',  
    body: []  
  }  
)
```

11、折叠器

说明：折叠面板，当信息量较大且分类较多时，可使用折叠面板进行分类收纳

示例

```
getSchemaTpl('collapseGroup',  
  [  
    {  
      title: '基本',  
      collapsed: true,  
      body: [  
        {  
          type: 'input-text',  
          label: '标题',  
          min: 0,  
          name: 'title',  
        }  
      ]  
    }  
  ]  
)
```

12、 api 配置

说明： 动态 api 接口面板

示例


```
getSchemaTpl('apiControl', {  
    name: 'dataSource.api',  
    visibleOn: '${dataSource.method == "api"}',  
    label: '接口设置',  
    mode: 'row',  
})
```

13、 提示

说明： 添加鼠标放置提示

示例

```
{  
    type: 'input-color',  
    name: 'xAxisStyle.nameTextStyle.color',  
    label: tippedLabel('单位颜色', '单位文字颜色'),  
    labelMode: 'input',  
}
```

14、 列配置

说明： 可以动态配置表格列，图表数据等

示例

```
{
  type: 'combo',
  name: 'columns',
  label: '表格列',
  multiple: true,
  multiLine: true,
  draggable: true, // 开启拖拽排序
  scaffold: {
    label: '列标题',
    value: 'col1',
    type: 'string'
  },
  items: [
    {
      type: 'input-text',
      name: 'label',
      label: '列标题'
    },
    {
      type: 'input-text',
      name: 'value',
      label: '字段名'
    },
    {
      type: 'select',
      name: 'type',
```

15、 flex 布局

说明： 水平布局排列

示例

```
{
  type: "flex",
  items: [
    {
      type: "container",
      body: [
        {
          type: "checkbox",
          name: "visibleAllChina",
          option: "显示南海群岛",
        },
      ],
    },
    {
      type: "container",
      body: [
        {
          type: "checkbox",
          name: "clickInnerRegion",
          option: "点击进入下级",
        },
      ],
    },
  ],
}
```

16、 日期

说明：年月日选择，支持相对值设定，如+2days 两天后

示例

```
{  
  "type": "input-date",  
  "label": "日期",  
  "name": "date"  
}
```

17、 滑块

说明：选择某个值或者某个范

示例

```
{  
  "type": "input-range",  
  "label": "滑块",  
  "name": "range"  
}
```

18、 下拉按钮

说明： 下拉按钮，更多的按钮通过点击后展示开来

示例

```
{
  "type": "dropdown-button",
  "label": "下拉按钮",
  "buttons": [
    {
      "type": "button",
      "label": "按钮 1"
    },
    {
      "type": "button",
      "label": "按钮 2",
    }
  ],
}
```

19、 键值对

说明： 用于编辑键值对类型的数据

示例

```
{
  "type": "input-kv",
  "draggable": true,
  "label": "KV",
  "name": "kv",
  "multiple": true,
  "items": [
    {
      "placeholder": "Key",
      "type": "input-text",
      "unique": true,
      "name": "key",
      "required": true,
      "validateOnChange": true
    },
    {
      "placeholder": "Value",
      "type": "input-text",
      "name": "value",  }
  ]
}
```

20、 勾选框

说明：配置是否开启某个功能

示例

```
{  
  "type": "checkbox",  
  "option": "勾选框",  
  "name": "checkbox",  
  "label": "勾选框",  
  "optionType": "default"  
}
```

4、 事件配置面板

自定义事件的配置面板，关键字段说明：

- label: 中文描述
- tag: 事件分类
- description: 事件描述
- supportComponents: 哪些组件可以调用此配置面板
- schema: 配置面板结构

完整示例如下：

```
export const MessageActionPanel = {  
  label: '自定义组件事件',  
  
  tag: '组件',  
  
  description: '组件事件',  
  
  supportComponents: ['my-renderer'],  
  schema: () => [  
    ...renderCmptSelect('目标组件', true),  
    {  
      type: "input-text",  
      label: "文本",  
      name: "args.text"  
    }  
  ],  
  descDetail: (info: any, context: any, props: any) => {  
    console.log(context, props);  
    return <div className="action-desc">{info?.label}1111</div>;  
  }  
}
```

5、样式文件(style.css)

样式应采用 CSS Modules 或具有良好作用域隔离的写法，避免全局污染。同时应支持通过 CSS 变量实现主题定制。

```
.smw-button {  
  // 使用 CSS 变量，便于主题定制  
  
  border-radius: var(--smw-button-radius, 6px);  
  font-family: var(--smw-font-family);  
  
  &--primary {  
    background-color: var(--smw-primary-color, #1890ff);  
  }  
}
```

样式变量

四、 公共 APIs

1、 路由

- Router 结构说明

API	函数签名	说明
getCurrentRoute	() => Route	获取当前解析后的路由信息，RouteLocation 结构详见下面说明
push	(target: string Route) => void	路由跳转方法，跳转到指定的路径或者 Route

replace	(target: string Route) => void	路由跳转方法，与 push 的区别在于不会增加一条历史记录而是替换当前的历史记录
open	(target: string Route) => void	新 Tab 页打开路由

● Route(路由信息) 结构说明

参数	说明	类型	可选值	默认值	备注
path	当前解析后的路径	String	-	-	必填
hash	当前路径的 hash 值，以 # 开头	String	-	-	必填
host	ip+端口	string	-	-	必填
query	当前的路径 query 对象	Object	-	-	必填，代表当前地址的 search 属性的对象
protocol	网络协议	http https	-	-	选填
fullPath	包括 search 和 hash 在内的完整地址	String	-	-	选填

● 使用示例

```
import { useRouter } from '@mse-lowcode/mse-widget-basis'

const router = useRouter()

// 获取路由信息

router.getCurrentRoute()
```

2、公共 Utils

- 获取当前环境信息

```
import { env } from '@mse-lowcode/mse-widget-basis'

// 获取当前用户
env.getCurrUser()

// 是否为编辑器模式
env.isEditor()

// 是否为预览模式
env.isViewer()
```

3、国际化

API	函数签名	说明
this.i18n	(i18nKey: string, params?: { [paramName: string]: string; }) => string	i18nKey 是语料的标识符，params 可选，是用来做模版字符串替换的。返回语料字符串
this.getLocale	() => string	返回当前环境语言 code
this.setLocale	(locale: string) => void	设置当前环境语言 code